VR2A-04-VARIS-(1)

VARIS OPTICAL DISC ARCHIVE HANDBOOK

SYSTEMS ENGINEERING

PROJECT VARIS · SELF-HOSTED ARCHIVE SYSTEMS

16 Sep 2026

RECORD OF REVISIONS/CHANGES

Rev Ltr	Chg No.	Description	Date
Basic		Design specification; data layer, burn pipeline and bin packing.	16 Sep 2026
	1	Cross-disc XOR parity, retrieval and group reconstruction; web UI and HTTP handlers.	17 Sep 2026
	2	Staged file listing; tags and description carried into the catalog.	17 Sep 2026
	3	Dry-run burn mode; media types gain ID prefix and write kind; per-burn target path.	17 Sep 2026
	4	Single WebDAV root for staging, retrieved and dryrun; collapsible staging folders.	17 Sep 2026
A		Pull request #1 merged; continuous integration builds and publishes the container image.	18 Sep 2026
A	5	This handbook issued, describing the system as of commit 2c60c60.	5 Nov 2026

TABLE OF CONTENTS

PREFACE v

1 INTRODUCTION 1-1
1.1 PURPOSE. 1-1
1.2 SCOPE AND APPLICATION. 1-1

2 DOCUMENTS 2-1
2.1 APPLICABLE DOCUMENTS. 2-1
2.2 REFERENCE DOCUMENTS. 2-1

3 SYSTEM OVERVIEW 3-1
3.1 ARCHITECTURE. 3-1
3.2 PACKAGES. 3-1
3.3 PROTECTION LAYERS. 3-1

4 CATALOG DATA MODEL 4-1
4.1 SCHEMA. 4-1
4.2 IDENTITY AND INTEGRITY. 4-1
4.3 SEARCH. 4-1
4.4 METADATA LIFECYCLE. 4-1

5 BURN PIPELINE 5-1
5.1 PLANNING. 5-1
5.2 STATES. 5-1
5.3 DISC SEQUENCE. 5-1
5.4 WRITE KINDS. 5-2
5.5 CROSS-DISC PARITY. 5-2

6 RETRIEVAL 6-1
6.1 STATES. 6-1
6.2 READING A DISC. 6-1
6.3 FAILURE CLASSIFICATION. 6-1
6.4 RECONSTRUCTION. 6-2

7 DESIGN SOLUTION 7-1
7.1 REQUIREMENTS. 7-1
7.2 DECISION ANALYSIS. 7-1
7.3 OPEN FINDINGS. 7-2

8 OPERATIONS 8-1
8.1 DEPLOYMENT. 8-1
8.2 MOUNTING THE SHARES. 8-1
8.3 BURNING. 8-1
8.4 RETRIEVING. 8-1
8.5 VERIFICATION. 8-1

A ABBREVIATIONS AND ACRONYMS A-1

B GLOSSARY B-1

C CATALOG DATA DICTIONARY C-1
C.1 GENERAL. C-1

D HTTP INTERFACE D-1
D.1 GENERAL. D-1

LIST OF FIGURES

3-1 Varis Deployment and Component Diagram . . . 3-1
3-2 Disc Image Layout and Cross-Disc Parity . . . 3-2
4-1 Catalog Class Diagram . . . 4-1
4-2 Metadata Lifecycle Sequence Diagram . . . 4-2
5-1 Burn Planning Process Flow . . . 5-1
5-2 Burn Job State Diagram . . . 5-1
5-3 Data Disc Burn Sequence Diagram . . . 5-2
6-1 Retrieval Job State Diagram . . . 6-1
6-2 Disc Read Sequence Diagram . . . 6-1
8-1 Continuous Integration Process Flow . . . 8-2

LIST OF TABLES

3-1 Package Responsibilities . . . 3-1
5-1 Write Kinds . . . 5-2
5-2 Burn Data . . . 5-2
6-1 Retrieval Failure Classification . . . 6-2
7-1 Varis Requirements . . . 7-1
7-2 D-1 Trade Matrix . . . 7-1
7-3 D-2 Trade Matrix . . . 7-2
7-4 D-3 Trade Matrix . . . 7-2
7-5 D-4 Trade Matrix . . . 7-2
7-6 Open Findings . . . 7-3
8-1 Environment Variables . . . 8-1
8-2 Operating Limits . . . 8-2
A-1 Abbreviations and Acronyms . . . A-1
B-1 Glossary . . . B-1
C-1 Catalog Columns . . . C-1
D-1 Routes . . . D-1

LIST OF BOXES

VARIS DESIGN KEYS . . . 3-2
DECISION D-1. ONE BINARY FOR WEB, WEBDAV AND WORKER . . . 7-1
DECISION D-2. PAD PARITY OPERANDS TO MEDIA CAPACITY . . . 7-2
DECISION D-3. IN-MEMORY JOB UNDER ONE MUTEX . . . 7-2
DECISION D-4. STANDARD LIBRARY HTTP ROUTING . . . 7-2
DECISION D-5. WODIM FOR OPTICAL WRITES . . . 7-2

This handbook describes Varis, a self-hosted system that archives files onto optical discs and finds them again. It is written for the operator who runs the system and for the engineer who changes it. It explains what each part does, why it was built that way, and how to operate it.

Every number in this handbook was checked against the source code at commit 2c60c60. Where the code and the earlier design documents disagree, the handbook follows the code and says so. Section 7.3 lists the places where the code does not yet do what its own documents promise.

The handbook does not cover the Go language, PostgreSQL administration, or the internals of par2, xorriso and wodim. Comments go to the repository as issues.

SECTION 1
INTRODUCTION

1.1 PURPOSE.

Varis moves files from a network share onto numbered optical discs and keeps a searchable catalog of where each file went. The one idea is this: every file is written to exactly one disc, that disc carries its own repair data, a group of discs can carry a spare disc made of their XOR, and the catalog names the disc to insert when the file is wanted again.

1.2 SCOPE AND APPLICATION.

The handbook covers the single Go binary *archive-core*, its PostgreSQL catalog, the burn and retrieval state machines, the web user interface (UI) and the Web Distributed Authoring and Versioning (WebDAV) shares. It applies to one operator on a trusted home network with one drive. Multi-drive operation, encryption, authentication and replacement of a reconstructed disc are outside the scope of the system and of this handbook. Abbreviations are listed in Appendix A and terms in Appendix B.

NOTE

Varis has no authentication. Anyone who can reach port 8080 can read, stage, burn and delete. Run it only on a trusted local network.

SECTION 2
DOCUMENTS

2.1 APPLICABLE DOCUMENTS.

The following documents form a part of this handbook to the extent specified herein. They are kept in the repository under docs/superpowers/specs/.

2026-09-16 Optical Disc Archive, Design Spec

2026-09-17 Staged File Listing and Metadata, Design Spec

2026-09-17 Dry-Run Burn Mode, Design Spec

2026-09-17 Generalized Media/Drive Abstraction, Design Spec

2026-09-17 Unified WebDAV Mount, Design Spec

README.md Varis, setup and usage

2.2 REFERENCE DOCUMENTS.

The following documents contain supplemental information.

ISO 9660 Volume and File Structure of CDROM for Information Interchange

RFC 4918 HTTP Extensions for Web Distributed Authoring and Versioning

par2cmdline Parity Volume Set Specification 2.0 tools

NASA/SP-2016-6105 NASA Systems Engineering Handbook, Rev 2 (structure of this handbook)

KSC-DF-107 Technical Documentation Style Guide (writing rules of this handbook)

SECTION 3
SYSTEM OVERVIEW

3.1 ARCHITECTURE.

Varis runs as two containers on one Linux host under Docker Compose (figure 3-1). The *archive-core* container holds the web server, the WebDAV server and both job managers in one process. The *archive-db* container runs PostgreSQL 16 and holds the catalog. The operator's workstation is a client only: it mounts the WebDAV root and opens the web pages. The optical drive is passed into *archive-core* as a device, which is why the host must run Docker natively on Linux.

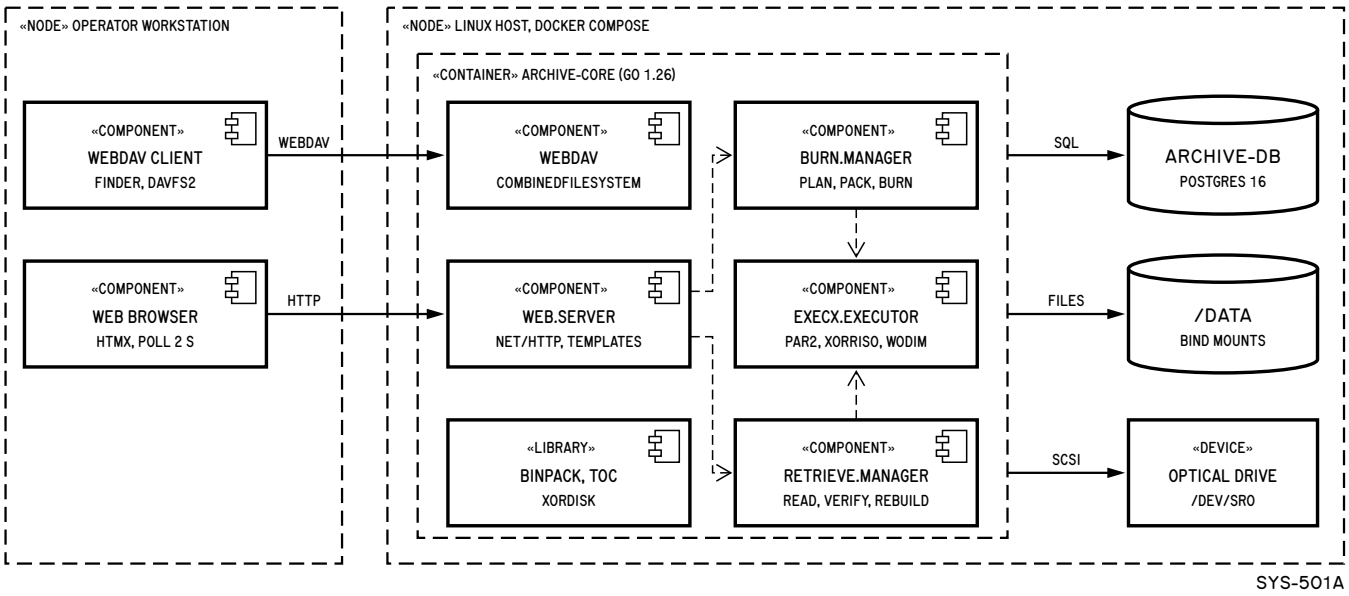


Figure 3-1. Varis Deployment and Component Diagram

3.2 PACKAGES.

The source is divided into Go packages with one job each (table 3-1). The two managers depend on the executor interface, not on *os/exec*, so the state machines run in tests without a drive or the external tools.

Table 3-1. Package Responsibilities

Package	Responsibility
cmd/archive-core	Loads configuration, connects to the catalog, seeds media types, mounts WebDAV, registers routes and listens.
internal/config	Reads seven environment variables with defaults (table 8-1).
internal/db	Embedded schema, applied on every start; queries for media types, disks, groups, files and staged metadata.
internal/binpack	Scans the staging tree and packs files into disc-sized buckets.
internal/toc	The TOC.json disc header: marshal and parse.
internal/burn	The burn state machine; tar, par2, ISO, write, verify, XOR parity and reconstruction.
internal/retrieve	The retrieval state machine, including group reconstruction.
internal/xordisk	One function: pad images to a length and XOR them together.
internal/execx	Executor interface over *os/exec*, and a fake for tests.
internal/webdav	One WebDAV root serving staging, retrieved and dryrun as folders.
internal/web	HTTP handlers and the embedded HTML templates.

3.3 PROTECTION LAYERS.

Each disc is protected twice (figure 3-2). Inside the disc, a par2 set sized at P percent of the payload guards against bit rot in part of the disc. Across discs, an optional parity disc holds the XOR of every data disc image in its group, so the loss of any one whole disc in the group can be repaired from the others.

3.3.1 Data Budget.

A data disc of capacity C at parity P receives at most C × 100 / (100 + P) bytes of files. A BD-R at 10 percent therefore takes 22,750,286,196 bytes of files.

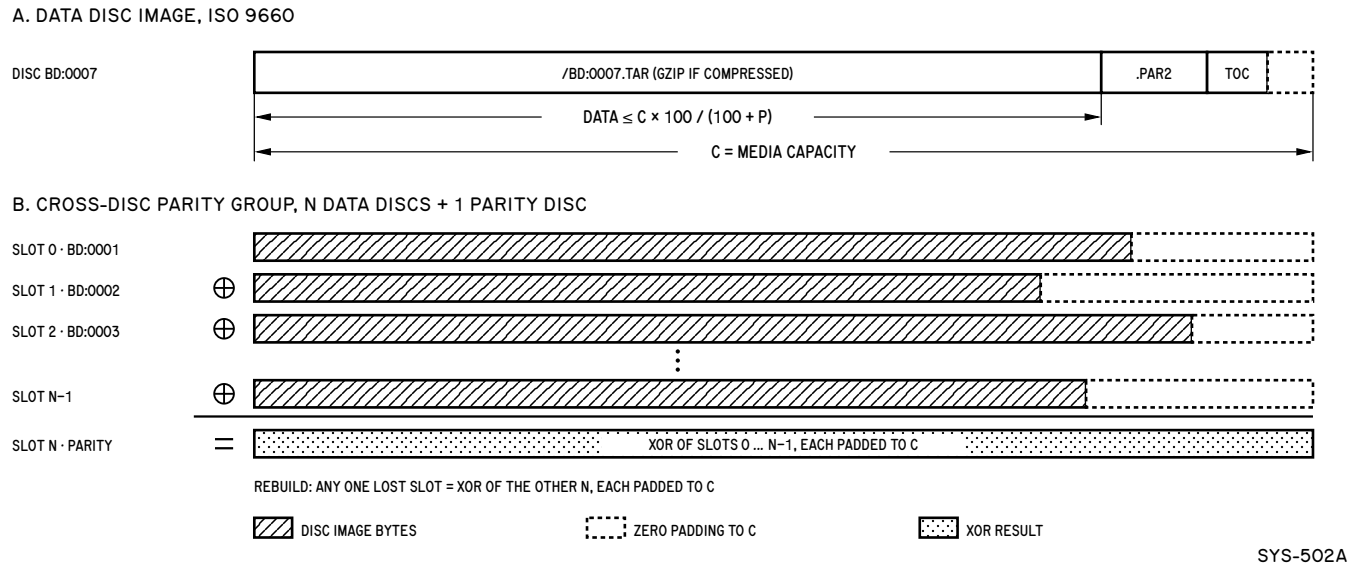


Figure 3-2. Disc Image Layout and Cross-Disc Parity

VARIS DESIGN KEYS

One drive, one job: a burn and a retrieval never run at the same time.
Nothing leaves staging until its disc has been written, read back, verified and cataloged.
Every disc can identify itself: TOC.json on the disc names its disk ID, media type, parity and group.
The catalog is the map, the disc is the territory: search the catalog, then insert the disc it names.
Every external tool runs through one interface, so every state transition can be tested without hardware.

SYS

SECTION 4
CATALOG DATA MODEL

4.1 SCHEMA.

The catalog is five PostgreSQL tables (figure 4-1). The schema file is embedded in the binary and executed on every start. Every statement in it is idempotent (CREATE ... IF NOT EXISTS, ADD COLUMN IF NOT EXISTS), so the same file creates a new catalog and migrates an old one. Appendix C lists every column.

4.1.1 Reading the Diagram.

Each table is drawn as a UML class with the stereotype «TABLE». Attributes read NAME : TYPE = DEFAULT; «PK» marks the primary key, «FK» a foreign key, and [0..1] a nullable column. Indexes sit in a third compartment. Multiplicities on the associations follow the nullability of the foreign key, not the intended use.

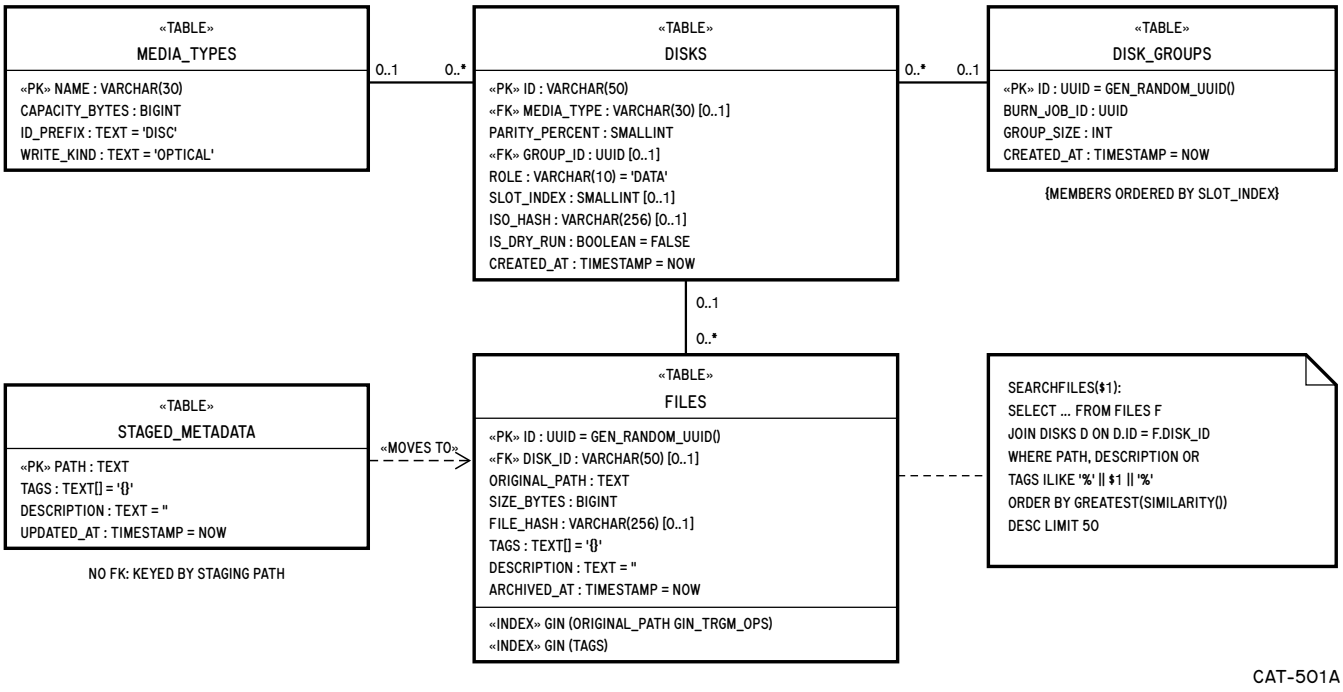


Figure 4-1. Catalog Class Diagram

4.2 IDENTITY AND INTEGRITY.

A disk ID is the media type's ID prefix, a colon and a four-digit sequence number, for example BD:0007. The next number is the count of existing disks with that prefix plus the number already handed out in the current plan, plus one. This is safe only because one job runs at a time; there is no sequence object or unique lock behind it. The string enumerations *role* (data, parity) and *write_kind* (optical, filesystem) carry no CHECK constraint; they are validated in Go where they are written. Disc and file rows are inserted without a transaction, one statement at a time.

4.3 SEARCH.

The Library page searches with one query (figure 4-1, note). It matches the search text as a case-insensitive substring of the path, the description, or the tags joined with spaces, and ranks rows by the greatest trigram similarity of the three. It returns at most 50 rows. Because the match is ILIKE with a leading wildcard over expressions, neither GIN index serves it; the query is a deliberate full scan, accepted at home-archive scale of tens of thousands of files.

4.4 METADATA LIFECYCLE.

Tags and a description can be attached to staged files before they are burned (figure 4-2). They are stored in *staged_metadata*, keyed by staging-relative path. When a file is cataloged, its row is deleted and returned in one statement, DELETE ... RETURNING, and the values are written into the new *files* row. The staged file is then removed from the staging share.

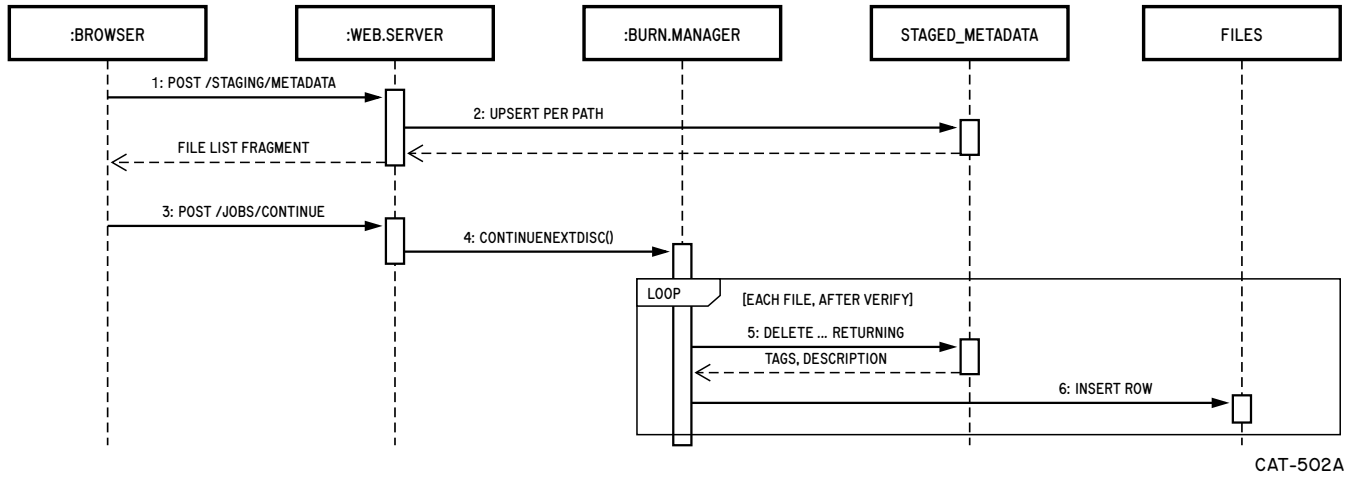


Figure 4-2. Metadata Lifecycle Sequence Diagram

CAUTION

Metadata is consumed before the file row is inserted. If the insert fails, that file's tags and description are lost and must be entered again; the file itself stays in staging and the disc can be retried.

CAT

SECTION 5
BURN PIPELINE

5.1 PLANNING.

Pressing BURN plans the whole job before any disc is written (figure 5-1). The planner first refuses the job if the spool directory has less than twice the media capacity free. It then walks the staging tree, sorts files by path, and fills discs in that order: a file that would push the current disc over its data budget (paragraph 3.3.1) starts a new disc. A single file larger than the budget gets a disc of its own. With cross-disc parity on, consecutive data discs are cut into groups of N, and each group, including a short last group, receives one parity disc.

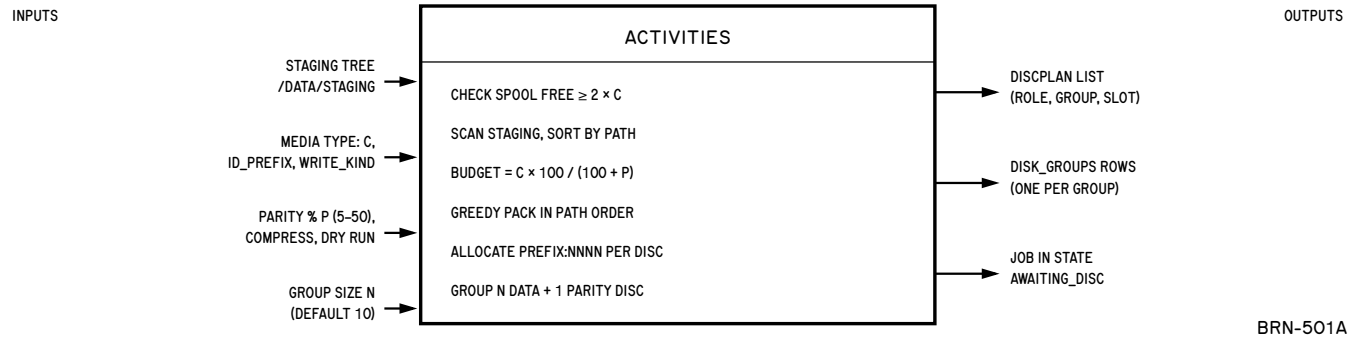


Figure 5-1. Burn Planning Process Flow

5.2 STATES.

The burn job is an in-memory state machine guarded by one mutex (figure 5-2). The job waits in AWAITING_DISC until the operator inserts a blank and presses continue. One disc plan then runs through PACKING, PARITY, ISO, BURNING and VERIFYING. A dry run copies the ISO to the dryrun share and skips VERIFYING. After the disc is cataloged the job returns to AWAITING_DISC for the next plan, or ends in DONE. Any error moves the job to FAILED with the error shown; retry returns it to AWAITING_DISC for the same plan without planning again.

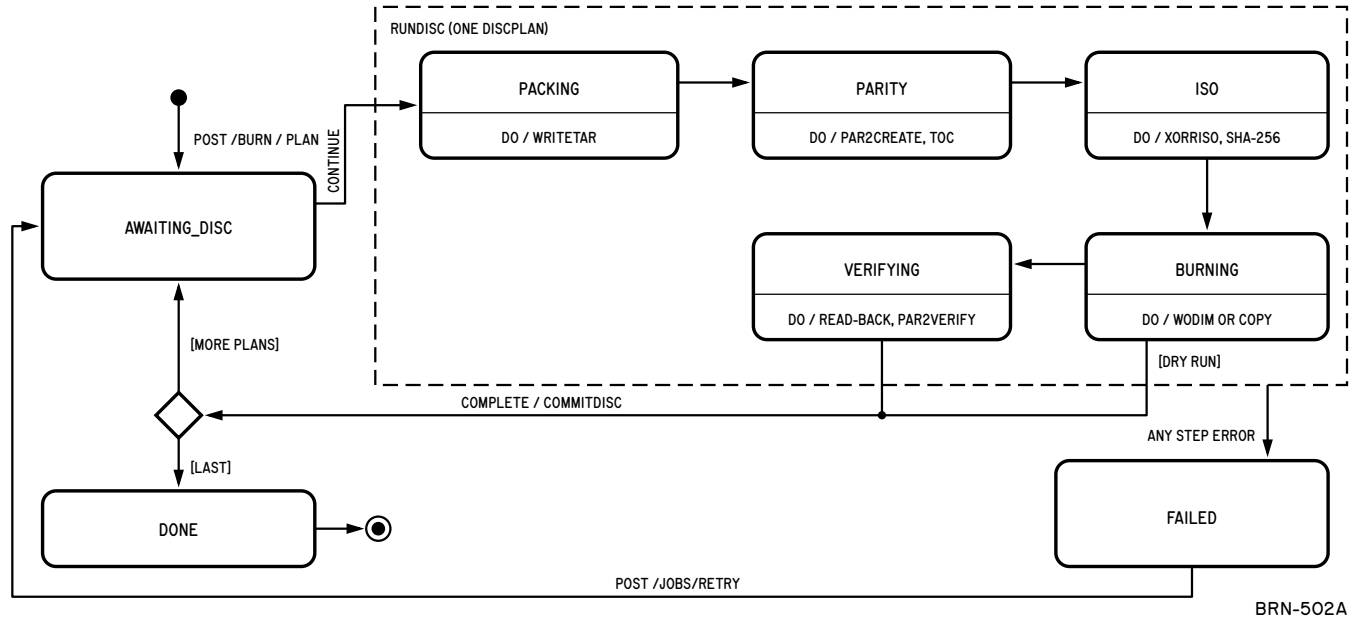


Figure 5-2. Burn Job State Diagram

5.3 DISC SEQUENCE.

Figure 5-3 shows one data disc on the optical write kind. The tar is written to the spool, par2 is created beside it, and the table of contents (TOC) is written. The three files are moved into a source folder and xorriso builds an ISO 9660 image with Rock Ridge and Joliet names. The image is hashed with SHA-256 before it is written. After wodim writes it, the device is read back and hashed; a mismatch fails the disc. par2verify then checks the payload. Only after both checks does the disc row go into the catalog, followed by one file row per file.

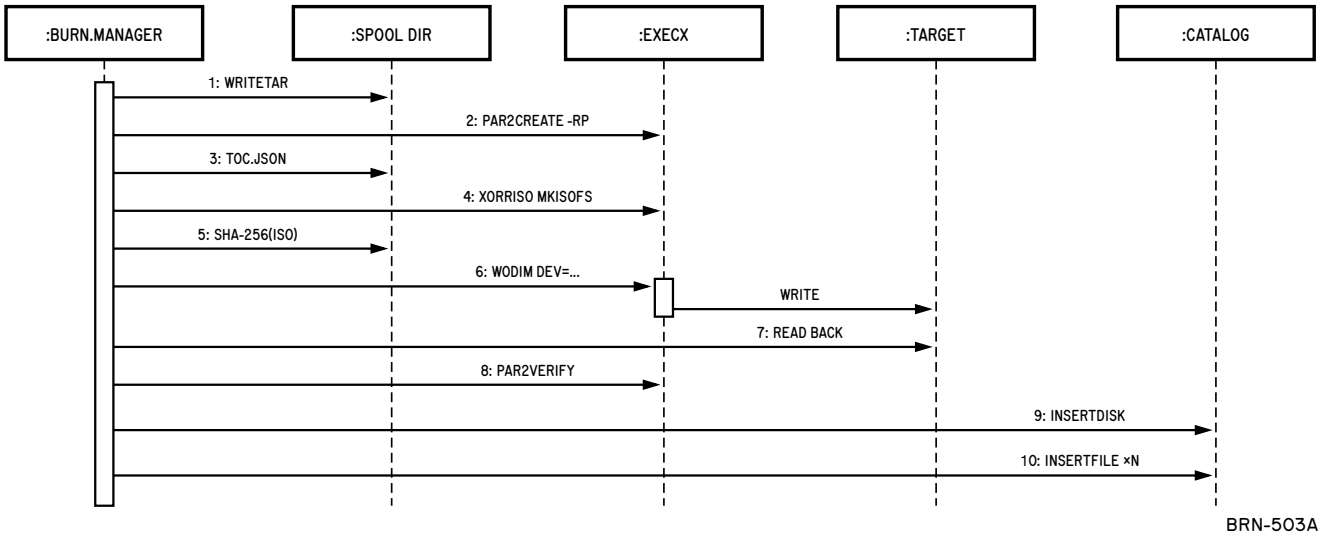


Figure 5-3. Data Disc Burn Sequence Diagram

5.4 WRITE KINDS.

How the image reaches the medium depends on the media type's write kind and the dry-run flag (table 5-1). The target path is typed into the dashboard for each burn and defaults to /dev/sr0.

Table 5-1. Write Kinds

Mode	Write	Verify
optical	wodim dev=<target> <iso>	SHA-256 of the device equals the ISO hash; par2verify
filesystem	Copy the ISO to the target path (RDX cartridge, USB drive)	As optical, then write <target>.sha256 in sha256sum format
dry run	Copy the ISO to /data/dryrun/<id>.iso; the disc row is flagged *is_dry_run*	None

5.5 CROSS-DISC PARITY.

A parity disc is built after the data discs of its group, in the same job. Its payload is the XOR of the group's data disc images, each read from the spool and zero-padded to the media capacity C (figure 3-2). The payload is then given its own par2 set and TOC, wrapped in its own ISO, and written and verified like a data disc. Once the parity disc is cataloged, the group's data images are deleted from the spool. Padding to C rather than to the largest image is decision D-2.

Table 5-2. Burn Data

Parameter	Value
Parity percentage P, accepted	5 to 50, default 10
BD-R capacity C	25,025,314,816 bytes
BD-R DL capacity C	50,050,629,632 bytes
BD-R data budget at P = 10	22,750,286,196 bytes
Spool free space required	2 × C, checked per disc
Group size N	default 10; values ≤ 1 become 10
Disk ID format	PREFIX:NNNN, e.g. BD:0007
Jobs page poll interval	2 s

SECTION 6
RETRIEVAL

6.1 STATES.

A retrieval is a second in-memory state machine with its own mutex (figure 6-1). Pressing GET in the Library opens a job in AWAITING_MEDIA that names the disc to insert. READ DISK reads it. The result is one of three: DONE, FAILED, or NEEDS_RECONSTRUCTION. Two short-lived states, STARTING_RECONSTRUCTION and READING_RECONSTRUCTION_DISC, exist only to claim the job while a request is working, so that two concurrent requests cannot both act on it.

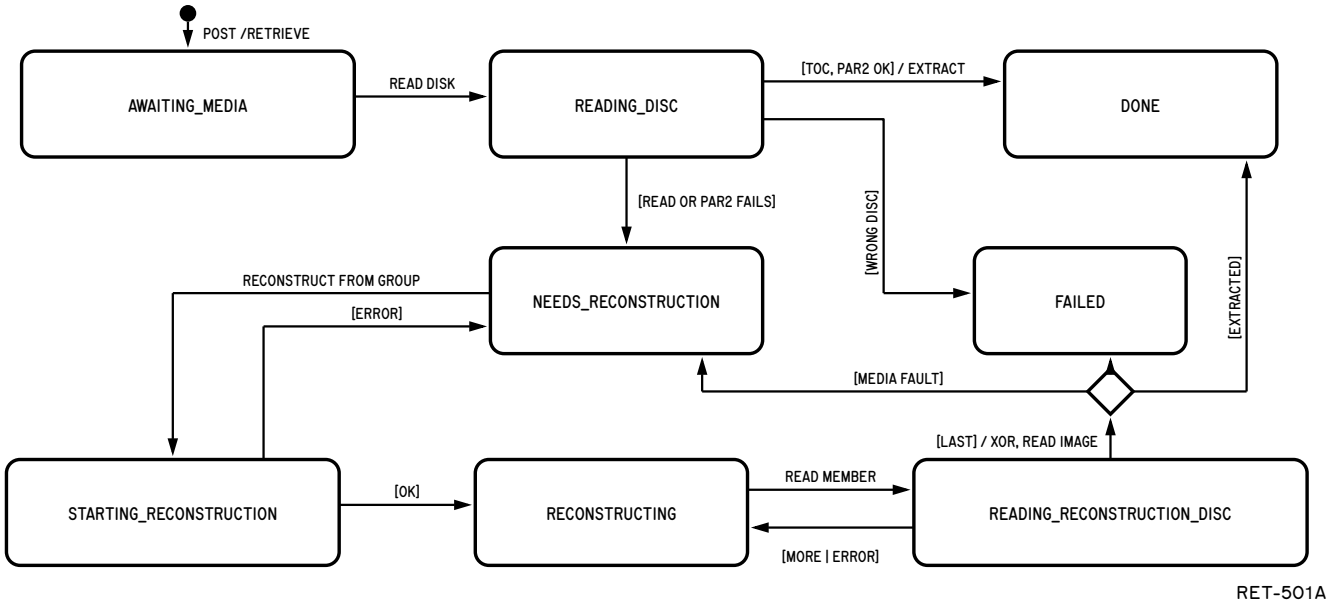


Figure 6-1. Retrieval Job State Diagram

6.2 READING A DISC.

A disc is read through xorriso, which accepts a device and an ISO file alike (figure 6-2). The manager extracts the TOC first and compares its disk ID with the one the catalog named. It then extracts the tar and its par2 index, runs par2verify, and extracts the one requested member of the tar into /data/retrieved, under the file's base name. A dry-run disc is read from /data/dryrun/<id>.iso; the target path is ignored.

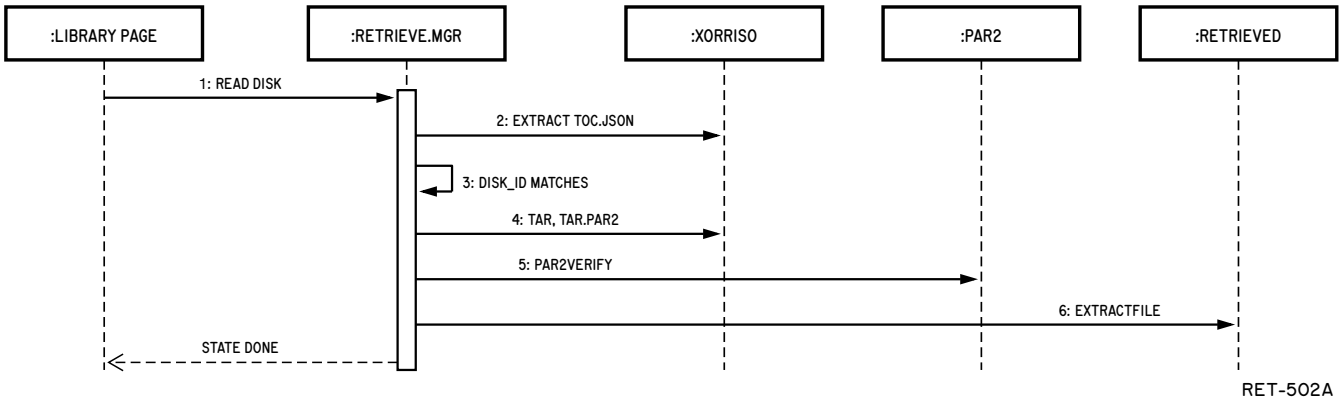


Figure 6-2. Disc Read Sequence Diagram

6.3 FAILURE CLASSIFICATION.

Every failure is sorted by what can fix it (table 6-1). A fault that may be damaged media leads to NEEDS_RECONSTRUCTION, because a group parity disc can repair it. A fault that reconstruction cannot repair leads to FAILED.

Table 6-1. Retrieval Failure Classification

Fault	Next State	Reason
TOC, tar or par2 index cannot be extracted; TOC does not parse	NEEDS_RECONSTRUCTION	The media may be damaged.
par2verify fails	NEEDS_RECONSTRUCTION	The payload is damaged beyond what the disc can repair.
TOC names a different disk ID	FAILED	The wrong disc is in the drive; insert the right one.
Member not found in a verified tar	FAILED	The tar is proven intact, so the file is not on this disc.
Catalog lookup fails	FAILED	No read was attempted.

6.4 RECONSTRUCTION.

If the disc belongs to a group, RECONSTRUCT FROM GROUP lists every other member of the group, data and parity, by disk ID. The operator inserts them in any order and presses READ for each. A data member is read raw from the device, exactly C bytes. The parity member is not read raw: its payload is extracted from inside its own ISO, because the raw parity disc is a wrapper around the XOR result, not the result itself. When the last member is supplied, the members are XORed into the missing disc's image, which is then read through the same path as a physical disc (paragraph 6.2). A failed member read leaves the job in RECONSTRUCTING with its progress kept.

CAUTION

Reconstruction repairs one lost disc per group. If two discs of the same group are lost, the files on both are unrecoverable.

SECTION 7
DESIGN SOLUTION

7.1 REQUIREMENTS.

The requirements below are the ones the code enforces today (table 7-1). Each is stated so that it can be verified: T by test, A by analysis, I by inspection.

Table 7-1. Varis Requirements

ID	Requirement	Rationale	Verif.
R-1	The system shall place at most C × 100 / (100 + P) bytes of files on one data disc, unless one file alone exceeds that.	Data plus par2 must fit the medium.	T
R-2	The system shall accept a parity percentage only as an integer from 5 to 50.	Below 5 % the repair data is too thin; above 50 % half the disc is parity.	T
R-3	The system shall run at most one burn or retrieval job at a time.	There is one drive.	T
R-4	The system shall refuse a burn disc when the spool has less than 2 × C bytes free.	The tar, par2 set and ISO coexist in the spool.	T
R-5	The system shall catalog a disc only after its read-back SHA-256 equals the ISO's and par2verify passes.	A disc that reads back wrong is not an archive.	T
R-6	The system shall remove a file from staging only after its file row is inserted.	A failed burn must lose nothing.	T
R-7	The system shall rebuild any one missing disc of a group from the other members of that group.	Total loss of one disc must not lose data.	T
R-8	A search shall return at most 50 rows.	The Library page lists results on one page.	I
R-9	Disk IDs shall be PREFIX:NNNN, sequential per prefix.	The ID is written on the disc cover and searched by hand.	T

DES

7.2 DECISION ANALYSIS.

The choices a newcomer is most likely to question are recorded below as decisions D-1 to D-5. The options and reasons come from the design specs and from comments in the code; where a reason is not recorded, the record says so. The scores in the trade matrices are this handbook's assessment against the criteria those sources name, on a scale of 1 to 3.

DECISION D-1. ONE BINARY FOR WEB, WEBDAV AND WORKER

STATUS. Accepted, 16 Sep 2026

CONTEXT. The web UI, the WebDAV server and the burn and retrieval workers all act on one drive and one job (requirement R-3). Design spec §2.

OPTIONS. One process; a web service and a worker service joined by a job queue; separate services sharing state through the catalog (table 7-2).

Table 7-2. D-1 Trade Matrix

Criterion (Weight)	One process	Queue	Shared DB
Fits a single drive and job (0.5)	3	2	2
Fewest moving parts (0.3)	3	1	2
Survives a process restart (0.2)	1	3	3
Weighted score	2.6	1.9	2.2

DECISION. Serve everything from one Go binary, *archive-core*.

RATIONALE. The parts share one in-process state machine and one drive, so splitting them buys nothing (spec §2).

CONSEQUENCES. Job state lives in memory: a restart forgets an in-flight job, though staged files and the catalog survive. A second drive would reopen the decision.

DECISION D-2. PAD PARITY OPERANDS TO MEDIA CAPACITY

STATUS. Accepted, 17 Sep 2026

CONTEXT. XOR needs operands of equal length. At reconstruction time the missing disc's real size is the one thing not known (requirement R-7). Comment on *buildParityPayload*.

OPTIONS. Pad every image to the media capacity C; pad to the largest image in the group and record that length (table 7-3).

Table 7-3. D-2 Trade Matrix

Criterion (Weight)	Pad to C	Pad to largest
Correct whichever disc is lost (0.6)	3	2
No extra state to record (0.3)	3	1
Smallest parity payload (0.1)	1	3
Weighted score	2.8	1.8

DECISION. Pad every operand to the fixed, known media capacity C.

RATIONALE. C is the only length both the burn side and the rebuild side know without the missing disc.

CONSEQUENCES. The parity payload is always C bytes, mostly zeros, and the whole group is held in memory while it is computed (see 7.3, F-2 and F-3).

DECISION D-3. IN-MEMORY JOB UNDER ONE MUTEX

STATUS. Accepted, 16 Sep 2026

CONTEXT. Only one job may run (requirement R-3). Design spec §7.

OPTIONS. An in-memory state machine guarded by a mutex; a job queue library (table 7-4).

Table 7-4. D-3 Trade Matrix

Criterion (Weight)	Mutex	Queue library
Matches one drive (0.5)	3	2
No new dependency (0.3)	3	1
Durable job state (0.2)	1	3
Weighted score	2.6	1.9

DECISION. Each manager keeps its job in memory behind a *sync.Mutex*; a state is claimed in the same critical section that checks it.

RATIONALE. One drive allows one job; a queue would have nothing to queue (spec §7).

CONSEQUENCES. Two managers mean two locks; the web layer checks one from the other (see 7.3, F-6).

DECISION D-4. STANDARD LIBRARY HTTP ROUTING

STATUS. Accepted, 16 Sep 2026

CONTEXT. About ten routes, each a method and a path. Design spec §2.

OPTIONS. *net/http* with Go 1.22 method and wildcard patterns; chi; gorilla/mux (table 7-5).

Table 7-5. D-4 Trade Matrix

Criterion (Weight)	net/http	chi	gorilla
No dependency (0.6)	3	1	1
Covers the routes needed (0.4)	3	3	3
Weighted score	3.0	1.8	1.8

DECISION. Route with *http.ServeMux* patterns such as "POST /burn".

RATIONALE. The route count does not justify a router dependency (spec §2).

CONSEQUENCES. Middleware such as authentication would be written by hand.

DECISION D-5. WODIM FOR OPTICAL WRITES

STATUS. Accepted, 16 Sep 2026

CONTEXT. The image must be written to BD-R media from inside a Debian container. Design spec §2.

OPTIONS. wodim; growisofs. The spec names no criteria beyond maintenance, so no trade matrix is given.

DECISION. Write images with wodim dev=<target> <iso>.

RATIONALE. The spec records wodim as the more actively maintained successor of cdrtools.

CONSEQUENCES. Burning depends on wodim's Blu-ray support on the host's drive; this is verified by hand on the Linux host, not in CI.

7.3 OPEN FINDINGS.

Writing this handbook required reading every code path against its documents. The findings in table 7-6 were confirmed in the code at commit 2c60c60. None of them affects a dry run; F-1 to F-3 affect real optical

media.

Table 7-6. Open Findings

ID	Finding	Effect
F-1	par2create writes an index <id>.tar.par2 and recovery volumes <id>.tar.vol...par2. Only the index is moved into the ISO source folder; the volumes stay in the spool and are not removed on retry.	The disc can detect damage but holds no recovery blocks to repair it. A retry may meet leftover volumes.
F-2	The parity payload is exactly C bytes before par2, TOC and ISO overhead are added.	A parity disc image is larger than the medium it is meant for; an optical write of it cannot succeed.
F-3	Parity build and reconstruction read whole images into memory.	Up to (N + 1) × C bytes of memory: about 275 GB for a group of 10 BD-R.
F-4	OPTICAL_DEVICE is loaded but never read.	The target is the form field, default /dev/sr0.
F-5	Compose does not mount /data/dryrun. README omits DRYRUN_DIR and the /dryruns routes and lists a PLANNING state the code does not have.	Dry-run images live in the container layer and are lost with it.
F-6	The burn and retrieval managers check each other without a shared lock.	Two requests arriving together can start both jobs.

SECTION 8
OPERATIONS

8.1 DEPLOYMENT.

Start the stack with docker compose up --build -d on a Linux host with the drive at /dev/sr0. Remove the comment marks from the *devices* block in docker-compose.yml before a real burn. Settings come from environment variables (table 8-1). The schema is applied and the two Blu-ray media types are seeded on every start. GET /healthz returns “ok”.

Table 8-1. Environment Variables

Variable	Default
DATABASE_URL	postgres://varis:varis@archive-db:5432/varis?sslmode=disable
STAGING_DIR	/data/staging
SPOOL_DIR	/data/spool
RETRIEVED_DIR	/data/retrieved
DRYRUN_DIR	/data/dryrun
HTTP_ADDR	:8080
OPTICAL_DEVICE	/dev/sr0 (not used, finding F-4)

8.2 MOUNTING THE SHARES.

Mount one WebDAV root, http://<host>:8080/webdav/. It holds three folders: *staging* (drop files here to archive them), *retrieved* (restored files appear here) and *dryrun* (dry-run images). The root itself is read-only; the three folders cannot be renamed or removed. On macOS use Finder, Go, Connect to Server.

8.3 BURNING.

Burn discs with the checklist below. Tagging is optional and only affects search. The job page asks for a blank disc before each disc of the plan.

OPS

CAUTION

Do not move or rename files in the staging share while a burn job exists. The plan holds paths, not copies; a missing file fails the disc at PACKING or when the file row is written.

BURN DISCS	OPERATOR
1 STAGING - copy files in	
2 APPLY TO SELECTED - tags (opt)	
3 MEDIA TYPE - select	
4 PARITY % - set 5 to 50	
5 BURN - press	
√ STATE - AWAITING_DISC	
6 BLANK DISC - insert	
7 CONTINUE - press, rpt per disc	
√ STATE - DONE	
8 PRINTABLE COVER - print	

8.4 RETRIEVING.

Retrieve a file with the first checklist. If the state becomes NEEDS_RECONSTRUCTION and the disc is in a group, continue with the second.

RETRIEVE FILE	OPERATOR
1 LIBRARY - search file	
2 GET - press	
√ STATE - AWAITING_MEDIA	
3 DISC <ID> - insert	
4 READ DISK - press	
√ STATE - DONE	
5 RETRIEVED - copy file out	

RECONSTRUCT DISC	OPERATOR
1 RECONSTRUCT FROM GROUP - press	
√ STATE - RECONSTRUCTING	
2 NEXT LISTED DISC - insert	
3 READ - press	
4 STEPS 2-3 - rpt per member	
√ STATE - DONE	

8.5 VERIFICATION.

Every push and pull request runs the checks in figure 8-1. Unit tests use the fake executor and in-memory catalogs, so the state machines are tested for their transitions without a drive. Catalog tests need PostgreSQL and skip when DATABASE_URL is unset; CI provides PostgreSQL 16 as a service. Real burns and reads are verified by hand on the Linux host.

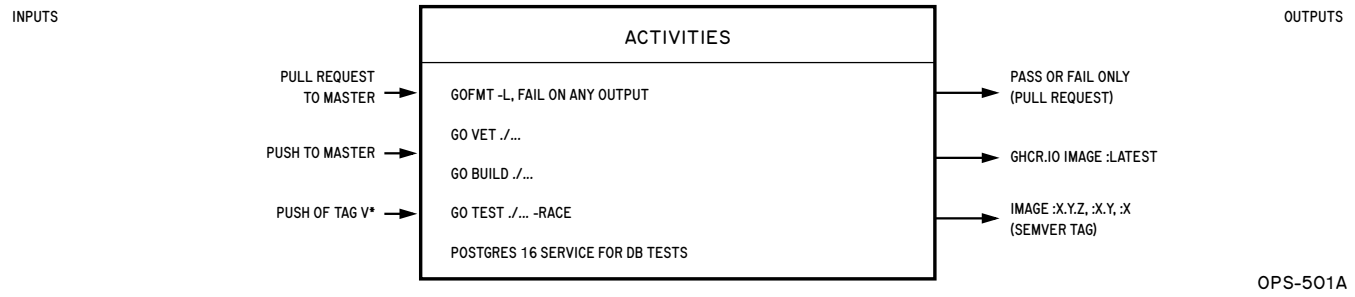


Figure 8-1. Continuous Integration Process Flow

Table 8-2. Operating Limits

Parameter	Value
Concurrent jobs, burn or retrieval	1
Drives	1 per job, path per request
Discs lost per group, recoverable	1
Search results per query	50
Library search keystroke delay	300 ms
Users	1, no authentication

APPENDIX A
ABBREVIATIONS AND ACRONYMS

Table A-1. Abbreviations and Acronyms

Term	Meaning
BD-R	Blu-ray Disc Recordable, 25 GB single layer
BD-R DL	Blu-ray Disc Recordable, dual layer, 50 GB
C	Media capacity in bytes, from *media_types.capacity_bytes*
CI	Continuous integration
DDL	Data definition language
FK	Foreign key
GIN	Generalized inverted index (PostgreSQL)
HTMX	A JavaScript library that swaps HTML fragments from server responses
ISO	ISO 9660 disc image
N	Number of data discs in a cross-disc parity group
P	Parity percentage of one burn
par2	Parity Volume Set 2.0, Reed–Solomon repair data for files
PK	Primary key
RDX	Removable disk cartridge system
SHA-256	Secure Hash Algorithm, 256-bit digest
TOC	Table of contents, the TOC.json disc header
UI	User interface
UML	Unified Modeling Language
WebDAV	Web Distributed Authoring and Versioning
XOR	Exclusive or

APPENDIX B
GLOSSARY

Table B-1. Glossary

Term	Definition
Bucket	The set of staged files planned for one data disc.
Data disc	A disc whose payload is a tar of staged files; role *data*.
Disc plan	One disc's planned work: disk ID, role, group, slot and bucket.
Disk ID	The catalog key of a disc, PREFIX:NNNN, printed on its cover.
Dry run	A burn that copies the image to the dryrun share instead of writing media.
Group	N consecutive data discs of one job plus their parity disc.
Parity disc	A disc whose payload is the XOR of its group's data disc images; role *parity*.
Slot	A disc's position in its group: 0 to N−1 for data, N for parity.
Spool	The working directory that holds a disc's tar, par2 set, TOC and ISO until it is done.
Staging	The WebDAV folder where files wait to be archived.
Write kind	How a media type is written: *optical* by wodim, *filesystem* by copy.

APPENDIX C
CATALOG DATA DICTIONARY

C.1 GENERAL.

Table C-1 lists every column of the catalog as created by internal/db/schema.sql. Null means the column accepts NULL. The class diagram is figure 4-1.

Table C-1. Catalog Columns

Column	Type	Null	Default	Notes
media_types				
name	varchar(30)	no		Primary key
capacity_bytes	bigint	no		C
id_prefix	text	no	'DISC'	BD and BDDL for the seeded types
write_kind	text	no	'optical'	optical or filesystem
disk_groups				
id	uuid	no	gen_random_uuid()	Primary key
burn_job_id	uuid	no		Ties the groups of one job
group_size	int	no		Data discs in this group
created_at	timestamp	yes	now	
disks				
id	varchar(50)	no		Primary key, PREFIX:NNNN
media_type	varchar(30)	yes		References media_types(name)
parity_percent	smallint	no		P of the burn
group_id	uuid	yes		References disk_groups(id)
role	varchar(10)	no	'data'	data or parity
slot_index	smallint	yes		Position in group
iso_hash	varchar(256)	yes		SHA-256 of the image, hex
is_dry_run	boolean	no	false	
created_at	timestamp	yes	now	
files				
id	uuid	no	gen_random_uuid()	Primary key
disk_id	varchar(50)	yes		References disks(id)
original_path	text	no		Staging-relative path; trigram GIN index
size_bytes	bigint	no		
file_hash	varchar(256)	yes		SHA-256 of the file, hex
tags	text[]	no	'{}'	GIN index
description	text	no	''	
archived_at	timestamp	yes	now	
staged_metadata				
path	text	no		Primary key, staging-relative path
tags	text[]	no	'{}'	
description	text	no	''	
updated_at	timestamp	yes	now	

APPENDIX D
HTTP INTERFACE

D.1 GENERAL.

Table D-1 lists every route registered in cmd/archive-core/main.go and internal/web/server.go. Routes that change state return an HTML fragment that HTMX swaps into the page.

Table D-1. Routes

Method	Path	Purpose
GET	/	Dashboard: staged files, burn form, job status
POST	/staging/metadata	Apply tags and description to selected files or folders
POST	/burn	Plan and start a burn job
GET	/jobs	Job status fragment, polled every 2 s
POST	/jobs/continue	Blank disc inserted; run the next disc plan
POST	/jobs/retry	Return a failed job to AWAITING_DISC
GET	/library	Search page
GET	/search?q=	Result rows, at most 50
POST	/retrieve	Start a retrieval for one file
POST	/retrieve/read	Read the named disc
POST	/retrieve/reconstruct/start	Begin group reconstruction
POST	/retrieve/reconstruct/read	Read one group member
GET, POST	/config	List and add media types
GET	/cover/{diskID}	Printable cover with a QR code for archive://disk/<id>
GET	/dryruns	List dry-run discs
POST	/dryruns/delete	Delete a dry-run image and its catalog rows
any	/webdav/	WebDAV root: staging, retrieved, dryrun
GET	/healthz	Returns “ok”